

Muzzamil Rasully ECS665U 210927089

Question 1 — TF-IDF Sparse Retrieval Baseline

My TF-IDF baseline achieved Recall@1 of 0.5476, Recall@20 of 0.8794, Recall@100 of 0.9173, and MRR of 0.6546. This made it a useful starting point, but it also showed the main limitation of sparse retrieval: it depends heavily on lexical overlap rather than meaning.

One failure example was “what is the shannara chronicles season 2 based on”, where the gold answer was “The Sword of Shannara Trilogy”. Instead, TF-IDF retrieved a passage about My Name Is Earl (season 4). This happened because words like “season” matched many television-related passages, so the system focused on surface-level word overlap rather than the actual meaning of the question.

A second failure was “what do you call initials that stand for something”, where the gold answer was “acronym”. TF-IDF retrieved an irrelevant passage about buildings in Las Vegas because the word “stand” matched unrelated text such as “stand taller”. This is a classic vocabulary mismatch — the answer, “acronym”, uses a word that never appears in the question, so TF-IDF had nothing lexical to match on.

TF-IDF worked best when the question shared exact names, dates or phrases with the gold passage. However, it struggled with paraphrasing, synonymy and vocabulary mismatch, which motivated the move to dense retrieval in Q2.

Question 2 — Dense Retrieval with Pre-trained Embeddings

For Q2, I used sentence-transformers/all-MiniLM-L6-v2 as my main bi-encoder. I chose this model because it is small, fast, and practical to run in Colab. It produces 384-dimensional embeddings, which keeps the index size smaller and makes retrieval more efficient. Since it is trained as a general sentence embedding model, it is also better suited than TF-IDF for capturing semantic similarity between questions and passages.

MiniLM clearly outperformed the TF-IDF baseline. TF-IDF achieved Recall@20 of 0.8794 and MRR of 0.6546, while MiniLM achieved Recall@20 of 0.9977 and MRR of 0.9537. Recall@1 also jumped from 0.5476 to 0.9208, which was a much larger gain than I expected from an off-the-shelf model with no NQ-specific fine-tuning.

I also compared MiniLM with the larger all-mpnet-base-v2 model. MPNet improved Recall@20 from 0.9977 to 1.0000, Recall@1 from 0.9208 to 0.9506, and MRR from 0.9537 to 0.9716. However, it used 768-dimensional embeddings and took around 10.8 seconds to encode the validation corpus, compared with around 2.5 seconds for MiniLM.

MPNet gave a small quality improvement, but at more than four times the encoding cost. For that reason, MiniLM stayed as my main model for the rest of the pipeline. Dense retrieval still has limitations because it can retrieve passages that are semantically similar but factually wrong. TF-IDF can still be useful for rare exact terms, dates, names, titles, and specific phrases, which is why I tested hybrid retrieval in Q3.

Question 3 — Advanced Retrieval Improvements with LLM Assistance

To get ideas for Q3, I used DeepSeek as a thinking partner to explore improvements beyond my dense MiniLM baseline. I started with a broad prompt explaining that I already had TF-IDF and dense retrieval, then asked for realistic Colab-friendly methods, including their intuition, tunable parameters, evaluation metrics and risks. This worked well because it led to the same techniques I implemented in the notebook: hybrid retrieval, cross-encoder re-ranking and query expansion.

The most useful prompts were the method-specific follow-ups. For weighted fusion, DeepSeek explained why TF-IDF and dense scores needed min-max normalisation before they could be combined. For RRF, it clarified that rank-based fusion avoids raw-score scale problems. For cross-encoder re-ranking, it explained why re-ranking only the top candidates is more practical than scoring the full corpus. These prompts helped shape my parameter searches for alpha, k_rrf, top_N and beta.

Some suggestions were useful conceptually but less practical for this coursework. Fine-tuning the retriever on Natural Questions could have improved performance, but it required more training time and GPU resources than were realistic in Colab. Query expansion was practical enough to implement, but the results showed that it did not improve retrieval. This was worth flagging because it is exactly the trap this question is testing for: the LLM’s confident suggestion still had to be validated against the actual development-set results.

In the experiments, I expected RRF to help because it combines TF-IDF and dense rankings without relying on raw scores. However, it actually performed worse than the dense baseline, with the best k_rrf=10 giving MRR 0.8361. Weighted fusion worked better: alpha=0.9 gave MRR 0.9545, only slightly above MiniLM’s 0.9537, showing that dense retrieval was already carrying most of the useful signal. The strongest improvement was cross-encoder re-ranking, where top_N=10 improved Recall@1 from 0.9208 to 0.9380 and MRR from 0.9537 to 0.9630. The cross-encoder won because it reads the question and passage side by side instead of comparing two pre-computed vectors. For ambiguous queries, that extra interaction is what makes the difference.

<i>Technique</i>	<i>Parameter search</i>	<i>Best setting</i>	<i>Main result</i>
<i>RRF hybrid retrieval</i>	k_rrf = 10, 30, 60, 100	k_rrf = 10	MRR = 0.8361
<i>Weighted score fusion</i>	alpha = 0.0–1.0	alpha = 0.9	MRR = 0.9545
<i>Cross-encoder re-ranking</i>	top_N = 10, 20, 50, 100	top_N = 10	MRR = 0.9630
<i>LLM query expansion</i>	beta = 0.5–1.0	beta = 1.0	MRR = 0.9537

Query expansion performed best at beta=1.0, meaning the original query alone was the strongest option. Inspecting the Flan-T5-small elaborations helped explain why: many were tautological or added little useful information, such as rewriting “who is the owner of the mandalay bay in vegas” as “the mandalay bay is owned by the owner of the mandalay bay in vegas”. The elaborations were genuinely unhelpful here — Flan-T5-small clearly did not know who owned the casino, so it repeated the question rather than adding useful context. This added noise without adding new retrieval signal, showing that a small generative model can fail to produce useful query expansions.

The takeaway was clear: when Recall@20 is already 0.9977, there is very little left to recover. The bigger gain comes from improving the top of the ranking, not from expanding the query. Therefore, I carried forward weighted hybrid retrieval with cross-encoder re-ranking.

Question 4 — Retrieval-Augmented Generation

I then extended the retrieval system into a RAG pipeline using FAISS as the vector database. I chose FAISS because it is fast, works well in Colab, and allows the index to be saved and reloaded without re-encoding the passages. Since my embeddings were normalised, I used IndexFlatIP, where inner product search behaves like cosine similarity.

I tested two prompt designs. Prompt A used a simple instruction to answer from the given context, while Prompt B added an explicit “I don’t know” fallback. Prompt A performed better, with Exact Match 0.2491 and Token F1 0.3580 at k=5, compared with Prompt B’s Exact Match 0.2239 and Token F1 0.3126. Prompt B also produced “I don’t know” in 186/871 cases, so it was too cautious and reduced answer coverage.

The k=1 result was counterintuitive because I expected more context to help rather than hurt. However, the best result was at k=1, with Exact Match 0.3238 and Token F1 0.4608. Performance dropped when k increased to 3, 5 or 10. This suggests that, after cross-encoder re-ranking, the top passage was usually strong enough on its own. With k=3 or more, the model first had to decide which passage to trust before extracting an answer. Flan-T5-small is small enough that this extra step seems to confuse it, especially when the retrieved passages cover related but distinct topics.

The retrieval-only baseline, which returned the top-1 passage text as the answer, achieved Exact Match 0.0000 and Token F1 0.1289. Full RAG improved performance because the generator produced concise answer strings rather than returning an entire passage. Generation is not free, though: when the model picks the wrong span or returns only half the answer, the F1 penalty is harsh.

Question 5 — Error Analysis and Final Evaluation

The final system combined MiniLM dense embeddings with a FAISS IndexFlatIP index, weighted hybrid fusion with $\alpha=0.9$, cross-encoder re-ranking at top_N=10, and Flan-T5-small using Prompt A with k=1. This achieved an Exact Match score of 0.3238 and a Token F1 score of 0.4608 on the validation set.

For the error analysis, I automatically classified all 589 Exact Match failures and then manually inspected 10 representative cases. The dominant failure type was answer span mismatch, which occurred in 504 cases (85.6%). Other errors included gold passages being retrieved but not passed to the generator in 45 cases (7.6%), partial matches in 40 cases (6.8%), hallucinations in 29 cases (4.9%), and retrieval misses in 2 cases (0.3%). This shows that the main bottleneck is not retrieval, but generation: the system usually retrieves relevant evidence, but Flan-T5-small often struggles to produce the exact gold answer string.

The inspected cases show this clearly. For “when did michigan last win a national championship”, the gold passage was ranked fourth, but because the final RAG setting used k=1, it was not passed to the generator, which instead answered “2012” from a rivalry-related

passage. For “who played taylor on the bold and beautiful”, the model predicted “Hunter Tylo”, but the gold answer required both “Hunter Tylo” and “Sherilyn Wolter”, making it a partial span error. The heatmap supports this interpretation: the gold Taylor passage had cosine similarities of around 0.46–0.56 with other soap-opera character passages, showing that the bi-encoder was grouping together highly similar character pages. The cross-encoder fixed some of these confusions — Recall@1 increased from 0.9208 to 0.9380 — but the soap-opera-style clusters remained a clear weakness. Finally, for “who sang i’m giving up on you”, the gold passage was missing from the top 20 because the system was distracted by a lexical overlap with “Give Up” as a Diana Ross track title.

With more compute, three changes would move the system forward. The first is a larger generator, such as Flan-T5-base or Flan-T5-large, which should help with the dominant span-mismatch problem. The second is contrastive fine-tuning of the bi-encoder using hard negatives, especially for clusters of very similar passages like the soap-opera character examples. The third is revisiting $k=3$ or $k=5$ once the generator is strong enough to handle multi-passage reasoning, because the current $k=1$ setting sometimes excludes the gold passage even when retrieval finds it.